

L'ARCHITETTURA **RISC-V** È UN'ARCHITETTURA **OPEN SOURCE**, PARLIAMO DELLA VERSIONE A **32 BIT**:

REGISTRI: IL PROCESSORE POSSIEDE **32 REGISTRI** ($X_0 - X_{31}$) CIASCUNO DA **32 BIT**.
IL REGISTRO X_0 CONTIENE SEMPRE IL VALORE **CONSTANTE ZERO**.

MEMORIA: L'INDIRIZZAMENTO AVVIENE AL SINGOLO **BYTE** (8 BIT), MA SI POSSO **LEGGERE/SCRIVERE** PAROLE INTERE DA **32 BIT**.
L'ORDINE DEI BYTE È **LITTLE ENDIAN**.
L'ACCESSO ALLA MEMORIA È PERMESSO SOLO DA ISTRUZIONI **LOAD** E **STORE**.

ISTRUZIONI:

ACCESSO ALLA MEMORIA:

- (**LOAD WORD**) **LW** $X_9, 8(X_{22})$ (CARICO LA PAROLA CHE STA ALL'INDIRIZZO $X_{22}+8$ E LO MEMORIZZO NEL REGISTRO X_9)
- (**STORE WORD**) **SW** $X_9, 24(X_{22})$ (SALVO IL CONTENUTO DI X_9 ALL'INDIRIZZO $X_{22}+24$)

ISTRUZIONI ARITMETICHE:

- **ADD** $X_9, X_{21}, X_9 \rightarrow X_9 = X_{21} + X_9$
- **ADDI** $X_9, X_9, 1 \rightarrow X_9 = X_9 + 1$ (X_9++)
- **SLLI** $X_{11}, X_{19}, 4 \rightarrow X_{11} = X_{19} \ll 4$ **SHIFT** DI **4 BIT** IL CONTENUTO DI X_{19}

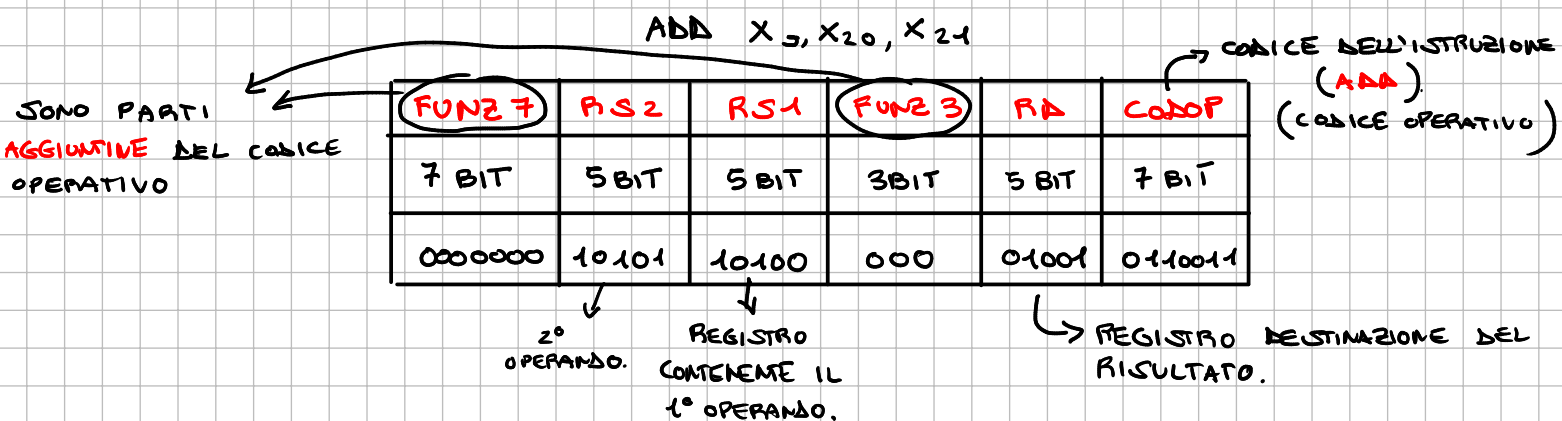
ISTRUZIONI DI CONTROLLO DEL FLUSSO:

- **BEQ** $X_{22}, X_{23}, \text{LABEL1} \rightarrow$ SE $X_{22} = X_{23}$ SALTA ALL'ISTRUZIONE CON ETICHETTA **LABEL1**
- **BNE** $X_{22}, X_{23}, \text{LABEL2} \rightarrow$ SE $X_{22} \neq X_{23}$ " " " " **LABEL2**

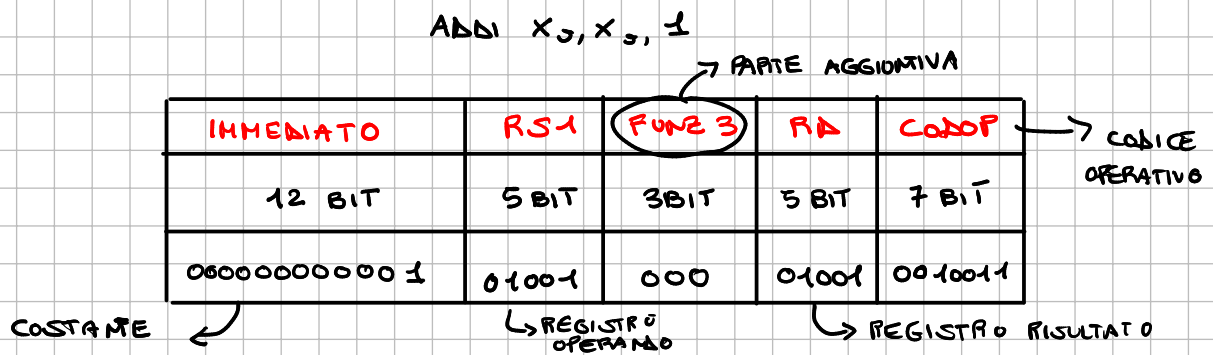
TUTTE QUESTE ISTRUZIONI SONO CODIFICATE IN **BINARIO** E DEVONO ESSERE IN MEMORIA (**RAM**) PER ESSERE **ESEGUITE**:

ES:

FORMATO R (ISTRUZIONI ARITMETICHE):



FORMATO I: OPERANDO IMMEDIATO (OP. ARITM.):



FORMATO I: OPERANDO IMMEDIATO (SHIFT)

SLLI X₁₁, X₁₀, 4

FUNZ 6	IMM.	RS-1	FUNZ 3	RA	CODOP
6 BIT	6 BIT	5 BIT	3 BIT	5 BIT	7 BIT
000000	000100	10011	001	01011	0010011

FORMATO I: OPERANDO IMMEDIATO (LOAD WORD)

LW X₅, 32(X₁₀)

IMMEDIATO	RS-1	FUNZ 3	RA	CODOP
12 BIT	5 BIT	3 BIT	5 BIT	7 BIT
000000100000	01010	010	01001	0000011

FORMATO S PER STORE IN MEMORIA (STORE WORD):

SW X₅, 32(X₁₀)

PRIMI 7 BIT + SIGNIFICATIVI
DELLA COSTANTE

IMMEDIATO [11:5]	RS 2	RS-1	FUNZ 3	IMMEDIATO [4:0]	CODOP
7 BIT	5 BIT	5 BIT	3 BIT	5 BIT	7 BIT
0000001	01001	01010	010	00000	0100011

REGISTRO
CONTENENTE
IL DATO
DA MEMORIZZARE

GLI ULTIMI
5 MENO SIGNIFICATIVI
DELLA COSTANTE

FORMATO SB: SALTO CONDIZIONATO (SI SALTA SOLO A INDIRIZZI PARI):

BNE X₁₀, X₁₁, 2000 (0011111010000)

IMMEDIATO [12,10:5]	RS 2	RS-1	FUNZ 3	IMMEDIATO [4:1,11]	CODOP
7 BIT	5 BIT	5 BIT	3 BIT	5 BIT	7 BIT
0111110	01011	01010	001	10000	1100111

IL BIT 12 PIÙ DAL 10 AL 5

14 BIT MENO SIGNIFICATIVI TRAMME
L'ULTIMO, PERCHÉ È SEMPRE PARI È
IMPLICITO CHE SIA 0.

ESEMPIO CODICE IN ASSEMBLER:

TRADUZIONE DEL CODICE IN C:

WHILE (SALVA[I] == K) I++;

L'ARRAY **SALVA** È UN ARRAY DI INTERI IL CUI INDIRIZZO A CIAMMO SI TROVI IN X_{25} .

USIAMO X_{22} PER **I** E X_{24} PER **K**:

CICLO: SUI $X_{10}, X_{22}, 2$ // $X_{10} = I \ll 2 = I \cdot 2^2$

ADD X_{10}, X_{10}, X_{25} // $X_{10} = \text{INDIRIZZO DI SALVA}[I]$

LW $X_9, 0(X_{10})$ // REGISTRO TEMP. $X_9 = \text{SALVA}[I]$

BNE X_9, X_{24}, ESCI // SE $\text{SALVA}[I] \neq K$ VAI A ESCI

ADDI $X_{22}, X_{22}, 1$ // $I++$

BEQ X_0, X_0, CICLO // VAI A CICLO

ESCI: ...

ESECUZIONE DI UN PROGRAMMA:

- **PRELEVA (FETCH)** L'ISTRUZIONE DALLA MEMORIA (INDIRIZZO IN PC).
- **AGGIORNA IL PC** PER PUNTARE ALLA PROSSIMA ISTRUZIONE.
- **DECODIFICA (DECODE)** L'ISTRUZIONE.
- **ESEGUE (EXECUTE)** (PUÒ MODIFICARE IL PC SE ISTRUZIONE DI SALTO).

LA CU SI OCCUPA DI **DECODIFICA L'ISTRUZIONE**, INVIA SEGNALI OPPORTUNI PER LA SUA ESECUZIONE E ASSICURA IL **CORRETTO AGGIORNAMENTO DEL PC**.

IL DATA PATH ESEGUE LE OPERAZIONI IN BASE AI **SEGNALI EMESSI** DALLA CU.

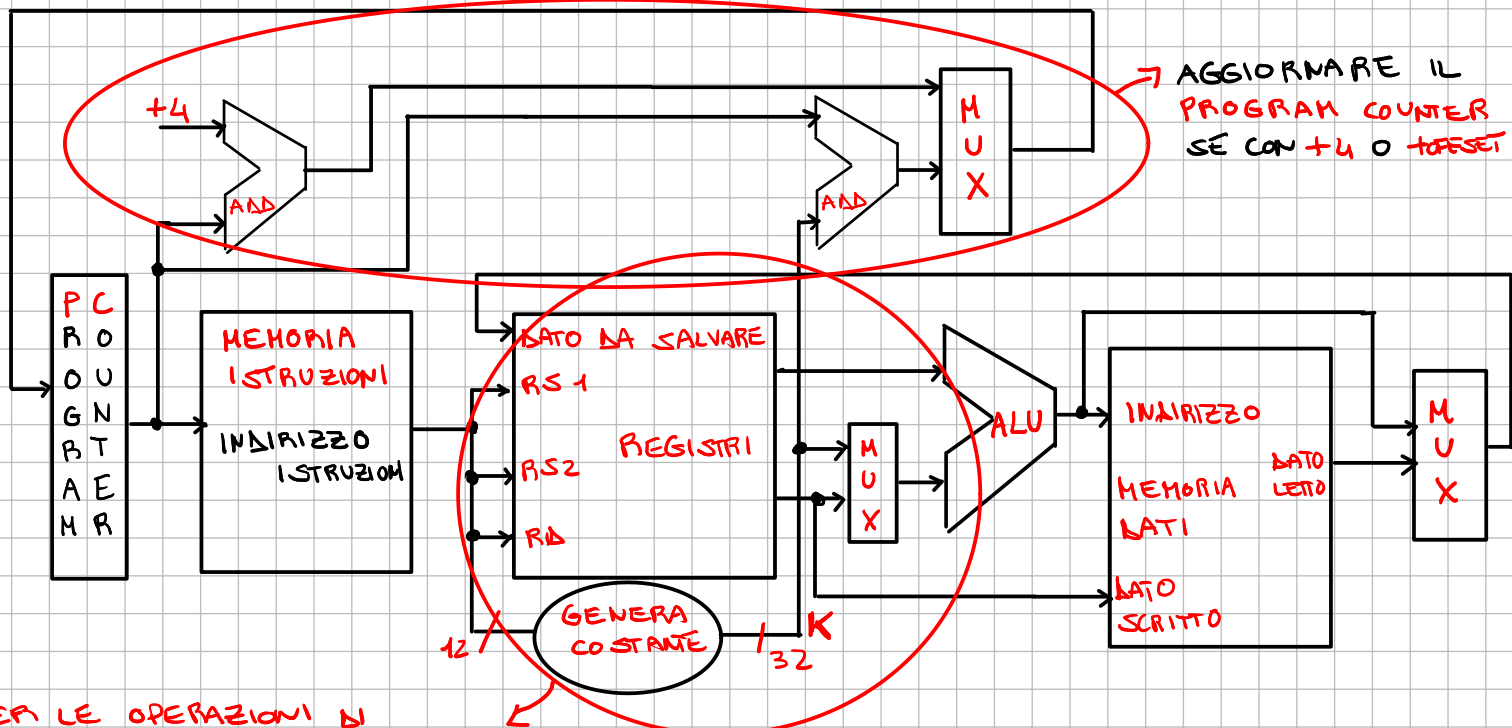
DECODIFICA: INTERPRETARE I DIVERSI **CAMPI DELL'ISTRUZIONE** (LE DIMENSIONI DELLE ISTRUZIONI E POSIZIONE DEI PARAMETRI SONO REGOLARI E PERMETTE UNA GENERAZIONE + **SEMPLICE** DEI SEGNALI).

ESECUZIONE: MOLTI PASSI IN COMUNE PER LE ISTRUZIONI ARITMETICHE, ACCESSO ALLA MEMORIA E SALTO CONDIZIONATO:

- **ALU**: PER OPER. ARITH., CALCOLO INDIRIZZO E VALUTAZIONE CONDIZIONE.
- **OPERANDI**: NEI REGISTRI O NELL'ISTRUZIONE (**CAMPO IMMEDIATO**).
- **AGGIORNAMENTO PC**: $+4$ SE NON SALTO, $+ \text{OFFSET}$ SE SALTO.

|
OGNI INTERO
OCCUPA 4 BYTE

CONSTRUZIONE DEL DATA PATH:



PER LE OPERAZIONI DI MEMORIA, L'ALU DEVE CALCOLARE L'INDIRIZZO ($RS1+K$) E SE:

- **LW**: DATO MEMORIZZATO IN **RD** (NEI REGISTRI)
- **SW**: VERRA' INVIATO IL DATO CONTENUTO IN **RS2** ALLA MEMORIA (LO LASCIA L)

VENGONO USATI DEI SEGNALI DI CONTROLLO COME:

- **OPERAZIONE ALU**: PER INDICARE QUALE OPERAZIONE FARE (DATO DA **CODOP**, **FUNZ7**, **FUNZ3**).
- **REGWRITE**: PER INDICARE CHE SI VUOLE SCRIVERE IL DATO IN ARRIVO NEL REGISTRO **RD**.
- **MEMWRITE / MEMREAD**: SEGNALE INVIATO ALLA MEMORIA SE SI VUOLE LEGGERE (**LW**) O SCRIVERE (**SW**).